

Desenvolvendo um módulo Magisk/Zygisk do zero

Conceitos, passo a passo e troubleshooting real — a partir do projeto FlutterTap

Eduardo Lopes
29 de julho de 2026

Tutorial construído a partir do desenvolvimento real do FlutterTap — um módulo Zygisk que intercepta o tráfego de apps Flutter. Cobre os conceitos, o passo a passo e, principalmente, todos os problemas que apareceram de verdade e como foram diagnosticados. A parte de troubleshooting é a mais valiosa: é onde se aprende o que nenhum tutorial costuma contar.

PARTE 1 — Conceitos que você precisa antes de escrever código

1.1 A pilha: root, Magisk, KernelSU, Zygisk

Root dá a você um processo com uid 0. Isso não é suficiente para modificar o comportamento de um aplicativo — um app roda no próprio processo, com a própria memória, e ninguém de fora simplesmente 'entra' nele. Precisa haver um mecanismo de injeção.

Magisk e KernelSU (e forks como SukiSu Ultra) são gerenciadores de root: montam modificações no sistema sem alterar a partição, e oferecem um sistema de módulos. APatch é outra alternativa. Todos eles resolvem 'como ter root e modificar arquivos do sistema'.

Zygisk é uma camada acima disso, e é o que interessa aqui: ele injeta código nativo em TODO processo de aplicativo, no momento em que ele nasce. No Magisk é um recurso embutido (precisa ser ativado nas configurações). No KernelSU/SukiSu não existe nativamente — é preciso instalar um módulo que o implemente, como Zygisk Next ou NeoZygisk.

1.2 Por que o zygote é o ponto de injeção

No Android, todo processo de app é criado por fork do zygote — um processo que já tem a máquina virtual e as bibliotecas do framework carregadas. Isso existe por desempenho: em vez de inicializar tudo a cada app, o sistema clona um processo já pronto.

A consequência é a oportunidade: se você conseguir estar presente no zygote, ou ser carregado em cada processo logo após o fork, você está dentro de todo app antes de a primeira linha de código dele rodar. É exatamente isso que o Zygisk faz, e é por isso que um módulo Zygisk consegue hookar coisas que acontecem na inicialização do app — algo difícil de conseguir anexando um depurador depois.

1.3 O ciclo de vida de um módulo Zygisk (isto vai importar muito)

Um módulo Zygisk implementa uma classe com callbacks. Os três que importam:

```
onLoad()          -> o modulo foi carregado (contexto varia! veja abaixo)
preAppSpecialize() -> antes do processo virar o app; ainda da para
                    falar com o processo companion (root)
postAppSpecialize() -> o processo ja e o app, sandbox aplicado
```

Atenção: O detalhe que causa mais bug: o CONTEXTO em que onLoad() roda depende da implementação do Zygisk. Com o carregamento tradicional, o módulo é carregado por processo, depois do fork — então onLoad() roda dentro do app. Mas o Zygisk Next tem um recurso (Zygisk Next Linker) que pré-carrega os módulos no PRÓPRIO zygote: aí onLoad() roda no zygote, e o que ele fizer é herdado por todos os apps do sistema. Nunca coloque em onLoad() nada que altere estado global do processo.

1.4 Escolhendo a ferramenta certa: módulo, Frida ou LSPosed

Frida é imbatível para descobrir como um app funciona: você edita JavaScript e vê o efeito em segundos. Mas depende de sessão ativa (fechou o terminal, perdeu o hook), não sobrevive a reboot, e abre uma porta em escuta com rastros bem conhecidos — detectar Frida é a primeira coisa que qualquer SDK de proteção mobile faz.

LSPosed/Xposed hooka métodos Java/ART. Se o que você quer interceptar é uma chamada do SDK do Android ou um método de uma classe Java, é a ferramenta certa. Se estiver em código nativo (como TLS dentro de uma .so), ele não alcança — você teria de fazer o hook nativo de qualquer forma, carregando o runtime do Xposed sem ganho.

Módulo Zygisk é o caminho quando você precisa de: código nativo, persistência entre reboots, operação sem ferramenta externa conectada, e presença desde o primeiro instante do app. O custo é iteração lenta — recompilar, reinstalar, reiniciar. A divisão prática: Frida para descobrir, módulo para operar.

PARTE 2 — Montando o projeto

2.1 Ferramentas

```
Android Studio (ou SDK + NDK avulsos)
NDK r27 ou compativel
CMake 3.22+ (vem com o SDK)
JDK 17 ou 21   <- NAO use 22+, veja troubleshooting
Git (dependencias nativas viram submodulos)
Um aparelho fisico com root + Zygisk ativo
```

Atenção: Emulador serve para desenvolver, mas o alvo real quase sempre é arm64-v8a. Compile as duas arquiteturas (arm64-v8a e x86_64) desde o começo: descobrir que o código só compila em uma delas depois de escrever tudo é desagradável.

2.2 Estrutura mínima de um módulo

Um módulo Zygisk é, no fim, um .zip com esta estrutura:

```
modulo.zip
|-- module.prop      metadados (id, nome, versao, autor)
|-- customize.sh     script de instalacao (opcional mas recomendado)
|-- uninstall.sh     roda na desinstalacao (opcional)
|-- action.sh        botao "Action" no gerenciador (opcional)
`-- zygisk/
    |-- arm64-v8a.so  sua biblioteca, um arquivo por arquitetura
    `-- x86_64.so
```

O gerenciador de root lê o module.prop, extrai os arquivos para /data/adb/modules/<id>/ e, no próximo boot, o Zygisk carrega a .so correspondente à arquitetura em cada processo de app.

```
id=meumodulo
name=Meu Modulo
version=v1.0.0
versionCode=1
author=Seu Nome
description=O que ele faz.
```

Atenção: O 'id' precisa ser estável e único: é o nome da pasta em /data/adb/modules e a chave que o gerenciador usa. Mudar o id depois equivale a criar um módulo novo, deixando o antigo instalado.

2.3 O header do Zygisk e a licença

Para compilar você precisa do header da API do Zygisk (zygisk.hpp). Um ponto de atenção jurídica que é fácil ignorar: use o header do repositório de exemplo oficial (topjohnwu/zygisk-module-sample), que é 0BSD. NÃO copie código do Zygisk Next — a licença dele é restritiva e proíbe redistribuição.

2.4 Ponto de entrada

```
class MeuModulo : public zygisk::ModuleBase {
public:
    void onLoad(zygisk::Api *api, JNIEnv *env) override {
        api_ = api; env_ = env;        // guarde e nada mais
    }

    void preAppSpecialize(zygisk::AppSpecializeArgs *args) override {
        // nome do processo: e assim que voce sabe QUAL app e este
        const char *nice = env_>GetStringUTFChars(args->nice_name, nullptr);
        processName_ = nice ? nice : "";
        env_>ReleaseStringUTFChars(args->nice_name, nice);

        shouldHook_ = ehAlvo(processName_);
        if (!shouldHook_) {
            // descarrega em todo app que nao interessa
            api_>setOption(zygisk::DLCLOSE_MODULE_LIBRARY);
        }
    }

    void postAppSpecialize(const zygisk::AppSpecializeArgs *args) override {
        if (!shouldHook_) return;
        // aqui o app ja e ele mesmo; faca o trabalho
    }
};

REGISTER_ZYGISK_MODULE(MeuModulo)
```

DLCLOSE_MODULE_LIBRARY é importante por dois motivos: reduz o impacto em apps que você não quer tocar (o módulo sai da memória) e reduz a superfície de detecção. Use sempre que o processo não for alvo.

2.5 Configuração: o processo companion

Seu módulo roda dentro do processo do app, com o sandbox do app — ele normalmente NÃO consegue ler /data/adb, onde faz sentido guardar configuração. A solução do Zygisk é o processo companion: um processo separado que roda como root irrestrito, com o qual você fala por socket.

```
// no companion (root): le o arquivo e devolve pelo socket
void companion_handler(int fd) {
    std::string json = leArquivo("/data/adb/meumodulo/config.json");
    uint32_t len = json.size();
    write(fd, &len, sizeof(len));
    write(fd, json.data(), len);
}
REGISTER_ZYGISK_COMPANION(companion_handler)

// no processo do app: SO funciona em preAppSpecialize
int fd = api_->connectCompanion();
std::string cfg = leDoSocket(fd);
close(fd);
```

Atenção: A conexão com o companion só é válida em preAppSpecialize, antes do sandbox. Se você tentar em postAppSpecialize, falha. E feche o descritor: o Zygisk reclama de FD vazado, e versões recentes do Zygisk Next chegam a relatar isso como erro do módulo.

Atenção: Guarde a configuração FORA da pasta do módulo (ex.: /data/adb/meumodulo/), não em /data/adb/modules/<id>/. Assim ela sobrevive a atualizações e reinstalações do módulo. E decida deliberadamente se o uninstall apaga ou preserva.

PARTE 3 — Fazendo o trabalho dentro do app

3.1 Esperar a biblioteca alvo carregar

Se o seu alvo é uma biblioteca que o app carrega durante a inicialização (não no momento do fork), ela ainda não existe em postAppSpecialize. Você precisa esperar — em uma thread, para não travar o app.

```
// thread de monitoramento: espera a lib aparecer
while (!encontraModulo("libalvo.so", mod)) {
    if (++tentativas > maxTentativas) return nullptr;
    usleep(50 * 1000); // 50 ms
}
fazTrabalho(mod);
```

Atenção: Para localizar a biblioteca, prefira `dl_iterate_phdr` a ler `/proc/self/maps`. Motivo prático: quando o APK é compilado com `extractNativeLibs=false` (o padrão moderno), a `.so` é mapeada direto de dentro do zip do APK, e o caminho em maps fica algo como `'.../base.apk!/lib/arm64-v8a/libalvo.so'` — um caminho sintético que não é um arquivo abrível. O `dl_iterate_phdr` informa a base de carregamento correta nos dois casos.

3.2 Encontrar funções não exportadas

Se a função que você quer hookar é exportada, `dlsym` resolve. O caso difícil — e comum — é quando ela não é. Aí o caminho é: achar uma referência conhecida (uma string, por exemplo) e navegar do código que a usa até o início da função.

1. delimitar as regiões: parsear os cabeçalhos ELF (`PT_LOAD`, `PT_GNU_RELRO`)
2. varrer a memória procurando uma string âncora (ex.: `"ssl_client"`)
3. achar a instrução que materializa o endereço dessa string
(arm64: `par adrp/add` | x64: `lea` relativo a RIP)
4. andar para trás até o prólogo da função
5. hookar esse endereço

Atenção: Erro clássico a evitar: usar padrões de BYTES fixos para achar as instruções. Parece funcionar e quebra silenciosamente em outra build, porque parte dos bits 'fixos' do padrão codifica QUAL REGISTRADOR o compilador escolheu — e outro compilador escolhe outro. Use um desmontador (Capstone é o mesmo que o Frida usa por baixo) e case por MNEMÔNICO, validando o endereço calculado. Fica agnóstico de registrador.

Toda leitura desse tipo é heurística: o endereço pode estar errado. Proteja-se — instale um handler de `SIGSEGV/SIGBUS` e use `setjmp/longjmp` para transformar uma leitura inválida em 'falhou' em vez de derrubar o app do usuário.

```
bool leituraSegura(void *dst, const void *src, size_t n) {
    if (sigsetjmp(estado->jmpBuf, 1) != 0) { estado->ativo = 0; return false; }
    estado->ativo = 1;
    memcpy(dst, src, n);
    estado->ativo = 0;
    return true;
}
```

3.3 Instalar o hook

Para hook inline em código nativo, Dobby é uma opção madura. Dois modos úteis: substituir a função inteira (guardando o ponteiro original para chamar depois) ou apenas inspecionar registradores na entrada.

```
// substitui: fakeFn roda no lugar, origFn guarda o original
DobbyHook(endereco, (void*)fakeFn, (void**)&origFn);

// so observa os registradores na entrada (nao substitui)
DobbyInstrument(endereco, meuCallback);
```

Atenção: Para hookar uma função da libc (socket, open...), NÃO use `dlsym(RTLD_DEFAULT, "socket")`. O `RTLD_DEFAULT` faz o bionic derivar o escopo de busca a partir da biblioteca CHAMADORA, mapeando o endereço de retorno para um registro do linker. Se o seu módulo foi carregado por um loader alternativo, esse registro não existe e `dlsym` devolve null. Use um handle explícito: `dlopen("libc.so", RTLD_NOW|RTLD_NOLOAD)` e depois `dlsym` nesse handle.

PARTE 4 — Empacotar, instalar, testar

4.1 Script de build

O empacotamento é simples, mas vale automatizar com verificações — foi o que evitou erro real:

1. compilar as duas ABIs
2. copiar as .so para stage/zygisk/<abi>.so
3. `llvm-strip --strip-unneeded` (reduz muito o tamanho)
4. VERIFICAR o binario (ex.: nenhuma relocation indesejada)
5. copiar module.prop e os scripts
6. zipar

Atenção: Inclua no zip as licenças das bibliotecas que você linka estaticamente. Apache-2.0 exige que a licença acompanhe distribuições binárias; BSD-3 exige reprodução do aviso de copyright. Se você linka Dobby (Apache-2.0) e Capstone (BSD-3) dentro da sua .so, essas obrigações valem para o seu .zip.

4.2 A sequência de teste que funciona

```
# 1. instalar e CONFERIR o checksum do que chegou
adb push modulo.zip /data/local/tmp/m.zip
adb shell su -c "ksud module install /data/local/tmp/m.zip" # ou magisk --install-module
adb shell su -c "sha256sum /data/adb/modules_update/<id>/zygisk/arm64-v8a.so"

# 2. reiniciar
adb reboot

# 3. SAUDE DO SISTEMA antes de testar funcionalidade
adb shell su -c "ps -A" | grep -c "^u0_a" # dezenas = ok, 0 = crash loop
adb shell su -c "pidof com.android.systemui" # vivo?
adb shell su -c "ls -lt /data/tombstones/ | head -3"

# 4. so agora o teste funcional
adb shell su -c "am force-stop <pacote>"
adb logcat -c
adb shell monkey -p <pacote> -c android.intent.category.LAUNCHER 1
adb shell su -c "logcat -d -s SuaTag:V"
```

Atenção: Por que checar saúde primeiro: um módulo defeituoso pode impedir que QUALQUER app inicie. Nesse estado o aparelho responde ao adb normalmente, mas fica com a tela preta — e você perde tempo investigando 'por que o hook não funcionou' quando nada consegue nem rodar.

Atenção: Instale um protetor de bootloop (ex.: Yet Another Bootloop Protector) no aparelho de teste e mantenha o SEU módulo FORA da allowlist dele. Assim, se o módulo quebrar o sistema, ele é desativado automaticamente e o aparelho volta utilizável — em vez de você ficar com um telefone inacessível.

PARTE 5 — Troubleshooting: os problemas reais

Esta é a parte que justifica o tutorial. Todos os casos abaixo aconteceram de verdade.

5.1 Ausência de crash em teste curto não prova nada

Uma correção foi declarada funcional após alguns minutos com o aparelho ligado e ocioso. O travamento real apareceu cerca de 40 minutos depois, quando apps começaram a ser abertos de fato.

Lição: a falha só se manifesta quando o CAMINHO DE FALHA executa. Exercite o caminho real — abrir, trocar, reabrir apps, deixar em uso — antes de concluir que corrigiu. E ao relatar, diga 'não verificado' até ter verificado.

5.2 Handler de sinal + DLCLOSE = todo app morre

Sintoma: aparelho liga, responde ao adb, mas fica com a tela preta. Investigando: nenhum processo de app rodando, SystemUI morto, e todo processo forkado pelo zygote morrendo com SIGSEGV imediatamente, em loop.

Causa: o módulo instalava um handler global de SIGSEGV/SIGBUS em onLoad(). Processos não-alvo então chamavam DLCLOSE_MODULE_LIBRARY, desmapeando a biblioteca — mas o handler continuava registrado no kernel, apontando para memória que acabara de ser liberada.

Por que era latente no Zygisk tradicional e fatal com o pré-carregamento no zygote: no primeiro caso o ponteiro pendurado ficava confinado a um app; no segundo, o onLoad() roda no zygote e disposições de sinal SOBREVIVEM AO FORK, então todo app nascia com o handler pendurado. E o ART levanta SIGSEGV benigno como parte da operação normal (verificação implícita de ponteiro nulo, página de guarda de stack overflow) — logo, todo app morria no primeiro.

Atenção: Regra geral que sai daí: nada que altere estado global do processo (handler de sinal, hook em libc, atexit, thread) deve ir em onLoad(). E se o módulo pode se descarregar, qualquer ponteiro que o processo guarde para dentro dele precisa ser removido antes.

5.3 dlsym(RTLD_DEFAULT) devolvendo null

Sintoma sutil: o módulo carregava, parte dos hooks instalava, mas um deles falhava silenciosamente — o tráfego continuava indo para o destino real enquanto os logs pareciam saudáveis.

Causa: RTLD_DEFAULT (e RTLD_NEXT) fazem o bionic derivar o escopo de busca da biblioteca chamadora, mapeando o endereço de retorno para um registro interno do linker. Um módulo mapeado por um loader alternativo não tem esse registro, a busca falha e dlsym devolve null. A correção é usar um handle explícito da libc.

Note o contraste útil: dl_iterate_phdr funcionava normalmente no mesmo cenário. O problema é específico das formas de dlsym que dependem do chamador.

5.4 Padrões de bytes que 'funcionam' e depois não

A resolução de endereços originalmente usava padrões de bytes com curingas para achar instruções. Funcionava na build de referência e falhava silenciosamente em outra, sem erro nenhum — só não

encontrava.

Causa: vários bits considerados 'fixos' no padrão eram, na verdade, a codificação do registrador que aquele compilador escolheu. Outro compilador, outro registrador, padrão não casa. A correção foi varrer por mnemônico com Capstone e validar o endereço computado — agnóstico de registrador.

5.5 Falha silenciosa: o pior tipo de bug

Vários problemas desta lista compartilham um padrão: o código escolhia um caminho padrão quando algo dava errado, sem registrar nada. Exemplos que corrigimos:

JSON de configuração inválido caía nos valores padrão, e o padrão tinha lista de alvos vazia — resultado: uma vírgula sobrando fazia o módulo simplesmente não fazer nada, sem uma linha de log. Arquivo de configuração ilegível: mesma coisa. Porta fora de faixa: convertida silenciosamente com truncamento (80833 virava 15297), e até o log mostrava o valor já truncado.

Atenção: Princípio: quando o código escolhe um padrão por causa de uma falha, registre. Uma linha de log em cada um desses pontos transformou 'instalei e nada acontece, não sei por quê' em diagnóstico de trinta segundos.

5.6 Retornar de um handler de SIGSEGV sem resolver a falha

Duas ramificações do nosso handler retornavam sem fazer nada. Retornar de um handler de SIGSEGV sem corrigir a causa faz a instrução faltosa ser REEXECUTADA — ou seja, o app travava a 100% de CPU em loop infinito em vez de crashar.

Detalhe extra de qualidade: para devolver o sinal ao dono anterior, restaurar a disposição e deixar a instrução faltar de novo é melhor que chamar `raise()`. O `raise()` gera um sinal `NOVO`, e o tombstone passa a apontar para o frame do seu handler — fazendo um crash alheio parecer culpa do seu módulo.

Atenção: Se você instala handler de SIGSEGV, adicione `SA_ONSTACK`. O ART instala o dele assim porque usa falha de página de guarda para detectar stack overflow; sem a flag, num overflow real o seu handler roda na pilha já esgotada.

5.7 Armadilhas de build e ambiente

JDK muito novo: o Gradle 8.11.1 não consegue interpretar a versão do JDK 22+ e falha com `IllegalArgumentException` antes de compilar qualquer coisa. Se o java padrão da máquina for novo, aponte `JAVA_HOME` para um JDK 17/21 (o JBR do Android Studio serve).

Git Bash no Windows e conversão de caminhos: o MSYS converte argumentos que parecem caminho Unix, corrompendo caminhos do Android. A variável `MSYS2_ARG_CONV_EXCL` desativa isso — mas exportá-la para o shell inteiro faz as ferramentas nativas do NDK receberem caminhos MSYS que elas não abrem, e o build falha. Aplique só ao comando do adb, ou converta com `cygpath` antes de chamar binários do Windows.

Fim de linha CRLF: os scripts do módulo (`customize.sh` etc.) rodam no sh do Android, que quebra com `\r`. O Git para Windows ativa `core.autocrlf` por padrão, então quem clonar no Windows recebe scripts com CRLF que vão para dentro do .zip. Resolva com um `.gitattributes` fixando `*.sh` em `eol=lf`.

Submódulo apontando para commit local: se você aplicar um patch em uma dependência que é submódulo e commitar só localmente, o `.gitmodules` continua apontando para o upstream — e `clone --recurse-submodules` falha para todo mundo, inclusive para você em outra máquina. Publique um fork ou versione o patch como arquivo.

5.8 Diagnóstico: o que ajuda e o que engana

`dmesg` é quase inútil para problema de boot: o buffer circular dá a volta em poucos minutos de uptime, então os logs do carregamento de módulos já foram sobrescritos quando você vai olhar. Use `tombstones (/data/tombstones/)`, que persistem e trazem backtrace e o campo 'Process uptime' — este último ajuda a separar 'falhou ao carregar' de 'morreu depois, em uso'.

O gerenciador de root costuma dar informação em tempo real: no KernelSU, o campo `description` do 'ksud module list' do módulo de Zygisk informa se o zygote foi injetado e, em versões recentes, reporta falha de carregamento de módulo. Confira isso antes de teorizar.

Atenção: A verificação de maior retorno por esforço: comparar o sha256 da .so instalada no aparelho com a do build local. Numa sessão real um build abortou silenciosamente, o comando de instalação enviou o pacote ANTERIOR, e o teste 'passou' validando código velho. Só o checksum revelou.

5.9 Cuidado ao ler documentação

Um diagnóstico inteiro foi construído sobre uma leitura errada de documentação: a wiki listava recursos 'não implementados', e três deles estavam TACHADOS (riscados) com link para a versão que os implementou. O tachado aparece no HTML renderizado, mas desaparece em qualquer conversão para texto puro — restando uma lista que parece dizer que nada funciona.

Custou um patch inútil e horas de investigação na direção errada. Lição: quando a documentação sustenta uma decisão importante, confira o formato original, não uma versão convertida. E prefira validar a hipótese com um experimento no aparelho antes de escrever a correção.

PARTE 6 — Checklist final

- [] `onLoad()` não altera estado global do processo
- [] processos não-alvo chamam `DLCLOSE_MODULE_LIBRARY`
- [] descritores do companion fechados antes de retornar
- [] toda leitura de endereço heurístico passa pelo guard de SIGSEGV
- [] nenhum `dlsym(RTLD_DEFAULT)` para símbolos de libc
- [] todo caminho de falha registra log
- [] valores de configuração validados (faixa, formato)
- [] licenças de terceiros dentro do .zip
- [] `.gitattributes` fixando *.sh em LF
- [] submódulos apontam para commits públicos
- [] testado nas duas ABIs
- [] testado com saúde do sistema verificada, não só com o hook
- [] testado por tempo suficiente, com apps sendo abertos de verdade

O item mais importante da lista é o último. A diferença entre um módulo que 'funciona na minha máquina' e um módulo confiável está quase toda em quanto tempo e sob que uso real ele foi observado.

— *Eduardo Lopes*